CS493 Final Project Proposal for Tarpaulin

Team Formation (Team 10)

- Calvin Chen
- Bailey Bounnam
- Duncan Everson

Team 10 proposes to implement the Tarpaulin API, a course management API similar to Canvas. The API will support users with admin, instructor, and student roles. Additionally, there will be courses, assignments, enrollments, student submissions, roster downloads, and plagiarism score processing.

Data Entities

users

Attributes:

- id (int, primary key, required): primary key of the user
- name (string, required): the name of the user
- email (string, unique, required): the email address of the user
- password_hash (string, required): the hashed password of the user
- role (string, enum, required): the role that the user has been assigned, which governs their permissions in the system.
    - Allowed enumerations
        1) "admin"
        2) "instructor"
        3) "student"

Relationships:

- A student can be enrolled in many courses through CourseStudent.
- An instructor can teach many courses through Course.instructor_id.
- A student can have many submissions.

courses

Attributes:

- id (int, primary key, required): primary key of the course
- subject_code (string, required): the subject code of the course

- number (int, required): the course number
- title (string, required): the title of the course
- term (string, required): the academic term for the course
- instructor_id (int, foreign key, required): the id of the instructor for the course

Relationships:

- A course belongs to one instructor.
- A course has many assignments.
- A course has many enrolled students through CourseStudent.

Unique Constraints:

- Only users with the instructor role should be assigned as a course instructor.

assignments

Attributes:

- id (int, primary key, required): primary key of the assignment
- title (string, required): the title of the assignment
- points (int, required): the amount of points assigned
- due_date (datetime, required): the due date for the assignment
- course_id (int, foreign key, required): the id of the course the assignment is associated with

Relationships:

- An assignment belongs to one course.
- An assignment has many submissions.

submissions

Attributes:

- id (int, primary key, required): primary key of the submission
- submitted_at (datetime, required): the time of the submission being made
- file_url (string, required): the URL leading to the location where the submitted file is stored
- plagiarism_score (int, nullable): the plagiarism score computed asynchronously after submission
- grade (int, nullable): the grade assigned to the submission after it is created
- assignment_id (int, foreign key, required): the id of the assignment the submission is associated with
- student_id (int, foreign key, required): the id of the student who made the submission

Relationships:

- A submission belongs to one assignment.
- A submission belongs to one student.

course_students (Pivot Table to Facilitate M:N for users and courses)

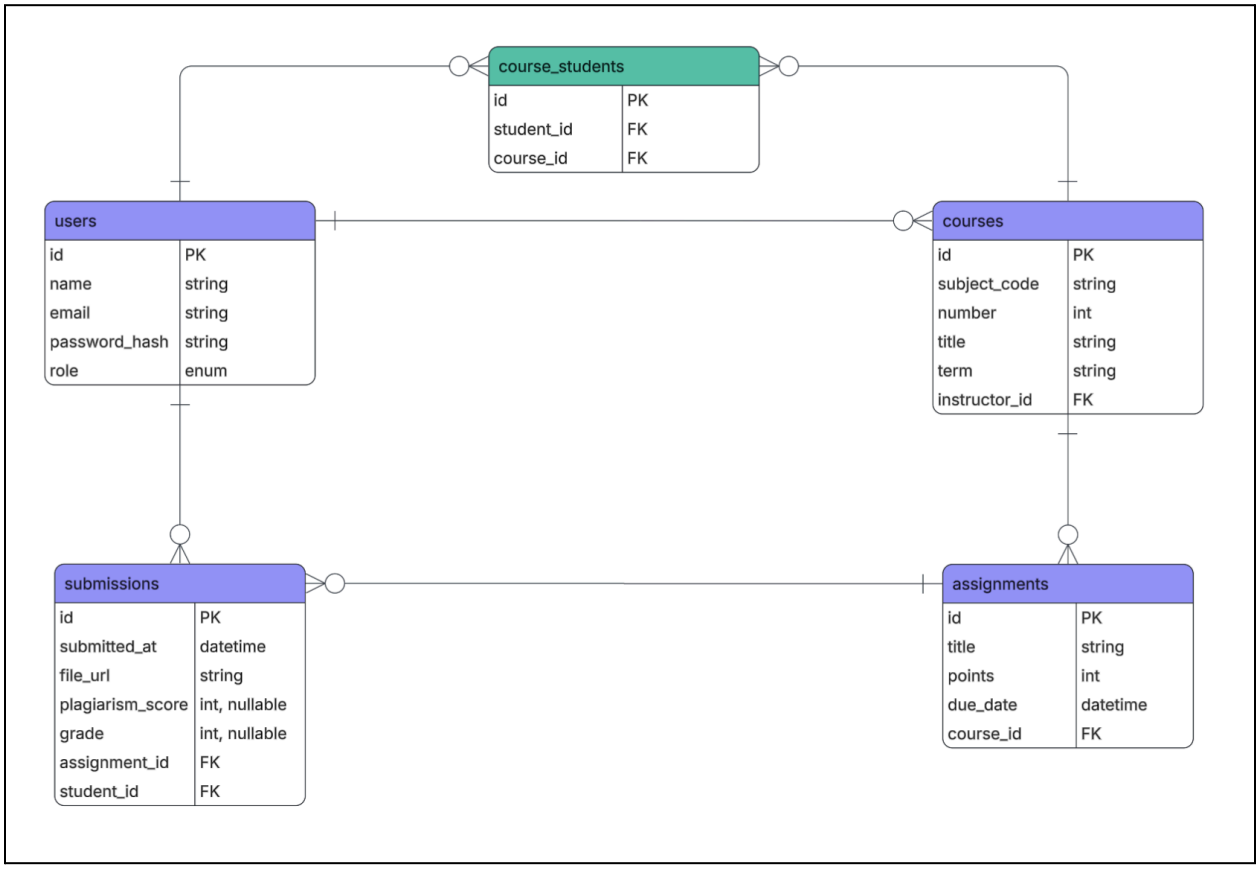
Attributes:

- id (int, primary key, required): primary key of student to course relationship
- student_id (int, foreign key, required): the id of the user associated with the course (will be a student)
- course_id (int, foreign key, required): the id of the course associated with a user

Unique Constraints:

- Only users with role = “student” are expected to be inserted into this table
- student_id + course_id pair must be unique

ER Diagram



Services to Power the API

Node.js / Express Backend:
The main API server will be built with Node.js and Express. It will handle routing, request validation, authentication, authorization, file uploads, and communication with the database.

Planned libraries:

- express
- prisma
- zod
- bcrypt
- jsonwebtoken
- multer
- file system
- bull (for Redis based job distribution)

PostgreSQL (will run in a Docker container):
PostgreSQL will be used as the main database. It will store users, courses, assignments, submissions, enrollments, grades, file URLs, and plagiarism scores.

Redis (will run in a Docker container):
Redis will be used to support offline plagiarism score processing. When a submission is created, the API will add a job to Redis. A worker will process the job later.

Background Worker:
A separate background worker will read submission jobs from Redis using the Node.js Bull library, send submitted files to the external plagiarism API, and update the corresponding submission record in PostgreSQL with the plagiarism score.

Docker:
Docker will be used to run all project services, including the Express API server, the background worker, PostgreSQL, and Redis.

Division of Labor

Calvin Chen: Focus on authentication, authorization, and user-related endpoints.
Responsibilities:

- Implement user login and JWT generation.
- Add password hashing with bcrypt.
- Create middleware for JWT verification.

- Implement role-based authorization for admins, instructors, and students.
- Implement GET /users/{id}.
- Help verify protected endpoints against the OpenAPI permission rules.

Bailey Bounnam: Focus on courses and enrollment.
Responsibilities:

- Implement course CRUD endpoints.
- Implement assignment CRUD endpoints.
- Implement filtering (by subject, number, and term) and pagination for GET /courses.
- Implement course enrollment through CourseStudent.
- Implement endpoints for fetching course students.
- Implement the course roster CSV download endpoint.
- Ensure course responses include required student and assignment counts.

Duncan Everson: Focus on assignments, submissions, and plagiarism processing.
Responsibilities:

- Implement submission upload using multipart form data.
- Store submitted files and generate file URLs.
- Implement paginated fetching of assignment submissions.
- Implement grade update functionality.
- Build the Redis-using worker for plagiarism score processing.

Shared Team Responsibilities
All team members will help with:

- Designing and updating the Prisma schema.
- Writing API tests for the grading demo.
- Testing authorization and error cases.
- Reviewing pull requests.
- Making sure Dockerized services run correctly.